

From NLTK to RoBERTa: A Comparative Analysis of Sentiment Analysis Techniques for Amazon Reviews

Introduction: This project delves into the world of Natural Language Processing (NLP) by building sentiment analysis models for Amazon reviews. I explored both traditional methods using NLTK and cutting-edge deep learning models like RoBERTa that's provided by hugging face. I used two main techniques the first one is the older kind of way of approaching sentiment analysis with a model called VADER (Valence Aware Dictionary and sentiment Reasoner) and this uses a bag of words approach and then used a pre-trained model from hugging face that's a roberta type model and this is a more advanced transformer model and I compared results between these two models and then I explored a hugging face pipeline. Through a comparative analysis, we gain a deeper understanding of NLP techniques for sentiment classification.

DATA: I took the Dataset from Kaggle and here is the link for the dataset <https://www.kaggle.com/datasets/snap/amazon-fine-food-reviews>.

I used this data set that is a bunch of amazon fine food reviews and these are text reviews for food on amazon as well as the rating that out of five stars that the reviewer gave them and all of this is in csv format.

Few rows of Dataset:

```
df.head()
```

	Id	ProductId	UserId	ProfileName	HelpfulnessNumerator	HelpfulnessDenominator	Score	Time	Summary	Text
0	1	B001E4KFG0	A3SGXH7AUHU8GW	delmartian	1.0	1.0	5.0	1.303862e+09	Good Quality Dog Food	I have bought several of the Vitality canned d...
1	2	B00813GRG4	A1D87F6ZCVE5NK	dll pa	0.0	0.0	1.0	1.346976e+09	Not as Advertised	Product arrived labeled as Jumbo Salted Peanut...
2	3	B000LQOCH0	ABXLMWJIXXAIN	Natalia Corres "Natalia Corres"	1.0	1.0	4.0	1.219018e+09	"Delight" says it all	This is a confection that has been around a fe...
3	4	B000UA0QIQ	A395BORC6FGVXV	Karl	3.0	3.0	2.0	1.307923e+09	Cough Medicine	If you are looking for the secret ingredient I...
4	5	B006K2ZZ7K	A1UQRSLF8GW1T	Michael D. Bigham "M. Wassir"	0.0	0.0	5.0	1.350778e+09	Great taffy	Great taffy at a great price. There was a wid...

Exploratory Data Analysis:



This gives us the number of times each score occurs and we can see here that most of the reviews are actually 5 stars but then it has a little uptick in the number of 1 star reviews, we have very biased towards positive reviews in our data set.

NLTK:

let's pick the 50th value as an example and we'll print this example

```
example = df['Text'][50]
print(example)
```

This oatmeal is not good. Its mushy, soft, I don't like it. Quaker Oats is the way to go.

Let's run NLTK word tokenizer

```
tokens = nltk.word_tokenize(example)
tokens[:10]
```

['This', 'oatmeal', 'is', 'not', 'good', '.', 'Its', 'mushy', ',', 'soft']

let's run nltk's pos tag for part of speech tagging and run this on each of these tokens

```
tagged = nltk.pos_tag(tokens)
tagged[:10]
```

```
[('This', 'DT'),
 ('oatmeal', 'NN'),
 ('is', 'VBZ'),
 ('not', 'RB'),
 ('good', 'JJ'),
 ('.', '.'),
 ('Its', 'PRP$'),
 ('mushy', 'NN'),
 (',', ','),
 ('soft', 'JJ')]
```

we can see that we have each token and we also have its part of speech and these part of speech values are codes and we can actually load up an example page that has some examples of what each abbreviation means so.

VADER Sentiment Scoring:

VADER stands for Valence Aware Dictionary and sentiment Reasoner, this approach essentially just takes all the words in our sentence and it has a value of either positive negative or neutral for each of those words and it combines up it just does a math equation and for all the words it'll add up to tell you how positive, negative or neutral that the statement is based on all those words.

One important thing is this approach does not account for relationships between words which in human speech is very important.

We will use NLTK's SentimentIntensityAnalyzer to get the neg/neu/pos scores of the text.

This uses a "bag of words" approach:

- 1) Stop words (words like “and”, “the” and words that really don't have a positive or negative feeling) are removed.
- 2) Each word is scored and combined to a total score.

let's run just on some examples

```
sia = SentimentIntensityAnalyzer()
```

```
sia.polarity_scores('I am so happy!')
```

```
{'neg': 0.0, 'neu': 0.318, 'pos': 0.682, 'compound': 0.6468}
```

```
sia.polarity_scores('This is the worst thing ever.')
```

```
{'neg': 0.451, 'neu': 0.549, 'pos': 0.0, 'compound': -0.6249}
```

```
sia.polarity_scores(example)
```

```
{'neg': 0.22, 'neu': 0.78, 'pos': 0.0, 'compound': -0.5448}
```

Running the polarity score on the entire data set so basically looping through this data frame and let's make a dictionary called res for results and every time we loop through we'll take my id and store in the my id part of the dictionary.

```
# Run the polarity score on the entire dataset
res = {}
for i, row in tqdm(df.iterrows(), total=len(df)):
    text = row['Text']
    myid = row['Id']
    res[myid] = sia.polarity_scores(text)
```

100%  500/500 [00:00<00:00, 780.81it/s]

```
vaders = pd.DataFrame(res).T
vaders = vaders.reset_index().rename(columns={'index': 'Id'})
vaders = vaders.merge(df, how='left')
```

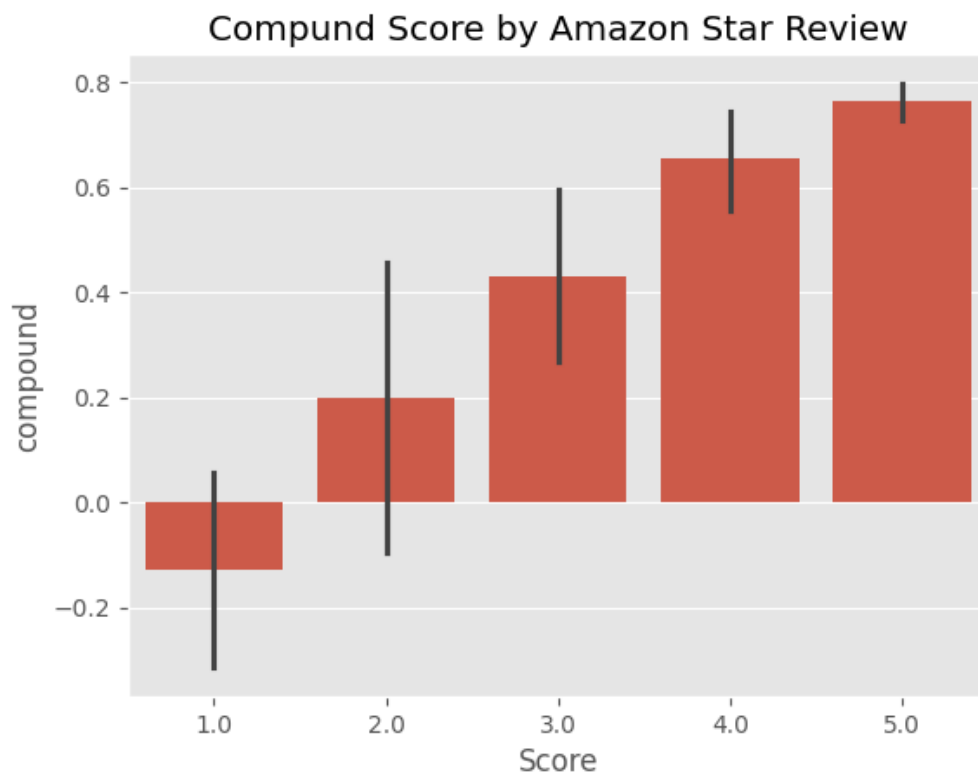
we can see now we have sentiment score and metadata

```
# Now we have sentiment score and metadata
vaders.head()
```

	Id	neg	neu	pos	compound	ProductId	UserId	ProfileName	HelpfulnessNumerator	HelpfulnessDenominator	Score	Time	Summary	Text
0	1	0.000	0.695	0.305	0.9441	B001E4KFG0	A3SGXH7AUHU8GW	delmartian	1.0	1.0	5.0	1.303862e+09	Good Quality Dog Food	I have bought several of the Vitality canned d...
1	2	0.138	0.862	0.000	-0.5664	B00813GRG4	A1D87F6ZCVE5NK	dll pa	0.0	0.0	1.0	1.346976e+09	Not as Advertised	Product arrived labeled as Jumbo Salted Peanut...
2	3	0.091	0.754	0.155	0.8265	B000LQOCH0	ABXLMWJXXAIN	Natalia Corres "Natalia Corres"	1.0	1.0	4.0	1.219018e+09	"Delight" says it all	This is a confection that has been around a fe...
3	4	0.000	1.000	0.000	0.0000	B000UA0QIQ	A395BORC6FGVXV	Karl	3.0	3.0	2.0	1.307923e+09	Cough Medicine	If you are looking for the secret ingredient I...
4	5	0.000	0.552	0.448	0.9468	B006K2ZZ7K	A1UQRSCLF8GW1T	Michael D. Bigham "M. Wassir"	0.0	0.0	5.0	1.350778e+09	Great taffy	Great taffy at a great price. There was a wid...

Bar plot of this data and our x value is going to be the score which is the star review of the person and then compound is going to be our y value

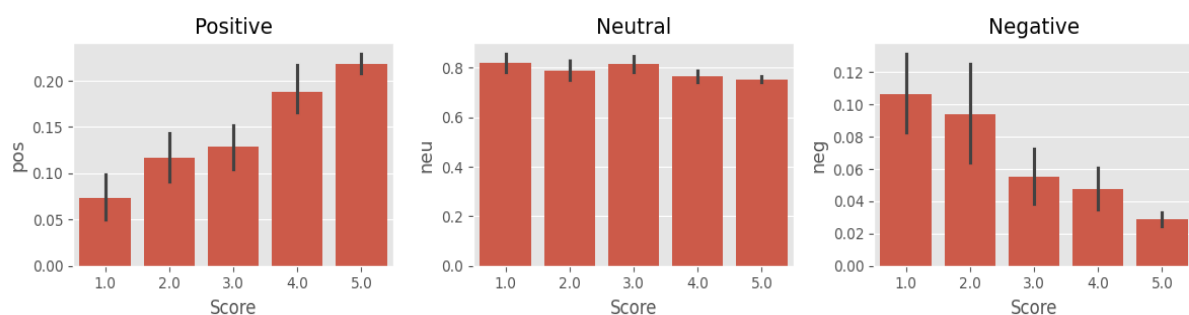
```
ax = sns.barplot(data=vaders, x='Score', y='compound')
ax.set_title('Compound Score by Amazon Star Review')
plt.show()
```



1 star review has lower compound score and a 5 star review is higher and it's actually exactly what we would expect.

Let's look at the positive neutral and negative scores for each using sns bar plot

```
fig, axs = plt.subplots(1, 3, figsize=(12, 3))
sns.barplot(data=vaders, x='Score', y='pos', ax=axs[0])
sns.barplot(data=vaders, x='Score', y='neu', ax=axs[1])
sns.barplot(data=vaders, x='Score', y='neg', ax=axs[2])
axs[0].set_title('Positive')
axs[1].set_title('Neutral')
axs[2].set_title('Negative')
plt.tight_layout()
plt.show()
```



we have the positivity higher as the score is higher in terms of stars, the neutral is kind of flat and the negative goes down this just confirms what we would hope to see.

RoBERTa Pretrained Model:

Our previous model just looked at each word in the sentence or in the review and scored each word individually but human language depends a lot on context if we see a sentence that could have negative words, actually it could be sarcastic or related to other words in which it makes it a positive statement so this VADER model wouldn't pick up on that sort of relationship between words.

Transformer based deep learning models have become very popular because they can pick up on that context so I took it from hugging face which is one of the leaders in these types of models and gathering them and making them easily available.

```
from transformers import AutoTokenizer
from transformers import AutoModelForSequenceClassification
from scipy.special import softmax
```

```
MODEL = "cardiffnlp/twitter-roberta-base-sentiment"
tokenizer = AutoTokenizer.from_pretrained(MODEL)
model = AutoModelForSequenceClassification.from_pretrained(MODEL)
```

```
/usr/local/lib/python3.10/dist-packages/huggingface_hub/utils/_token.py:88: UserWarning:
The secret 'HF_TOKEN' does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
config.json: 100%  747/747 [00:00<00:00, 13.2kB/s]
vocab.json: 100%  899k/899k [00:00<00:00, 3.75MB/s]
merges.txt: 100%  456k/456k [00:00<00:00, 2.51MB/s]
special_tokens_map.json: 100%  150/150 [00:00<00:00, 3.31kB/s]
pytorch_model.bin: 100%  499M/499M [00:03<00:00, 199MB/s]
```

VADER results on example

```
# VADER results on example
print(example)
sia.polarity_scores(example)
```

```
This oatmeal is not good. Its mushy, soft, I don't like it. Quaker Oats is the way to go.
{'neg': 0.22, 'neu': 0.78, 'pos': 0.0, 'compound': -0.5448}
```

RoBERTa model results

```
# Run for Roberta Model
encoded_text = tokenizer(example, return_tensors='pt')
output = model(**encoded_text)
scores = output[0][0].detach().numpy()
scores = softmax(scores)
scores_dict = {
    'roberta_neg' : scores[0],
    'roberta_neu' : scores[1],
    'roberta_pos' : scores[2]
}
print(scores_dict)
```

```
{'roberta_neg': 0.97635514, 'roberta_neu': 0.020687465, 'roberta_pos': 0.0029573692}
```


Roberta model thinks much more than the vader model that this comment is negative, from reading it seems to make sense this is a very negative review. we can see sort of how much more powerful Roberta is than just a vader model

Vader results and Roberta results:

```
def polarity_scores_roberta(example):
    encoded_text = tokenizer(example, return_tensors='pt')
    output = model(**encoded_text)
    scores = output[0][0].detach().numpy()
    scores = softmax(scores)
    scores_dict = {
        'roberta_neg' : scores[0],
        'roberta_neu' : scores[1],
        'roberta_pos' : scores[2]
    }
    return scores_dict
```

```
res = {}
for i, row in tqdm(df.iterrows(), total=len(df)):
    try:
        text = row['Text']
        myid = row['Id']
        vader_result = sia.polarity_scores(text)
        vader_result_rename = {}
        for key, value in vader_result.items():
            vader_result_rename[f"vader_{key}"] = value
        roberta_result = polarity_scores_roberta(text)
        both = {**vader_result_rename, **roberta_result}
        res[myid] = both
    except RuntimeError:
        print(f'Broke for id {myid}')
```

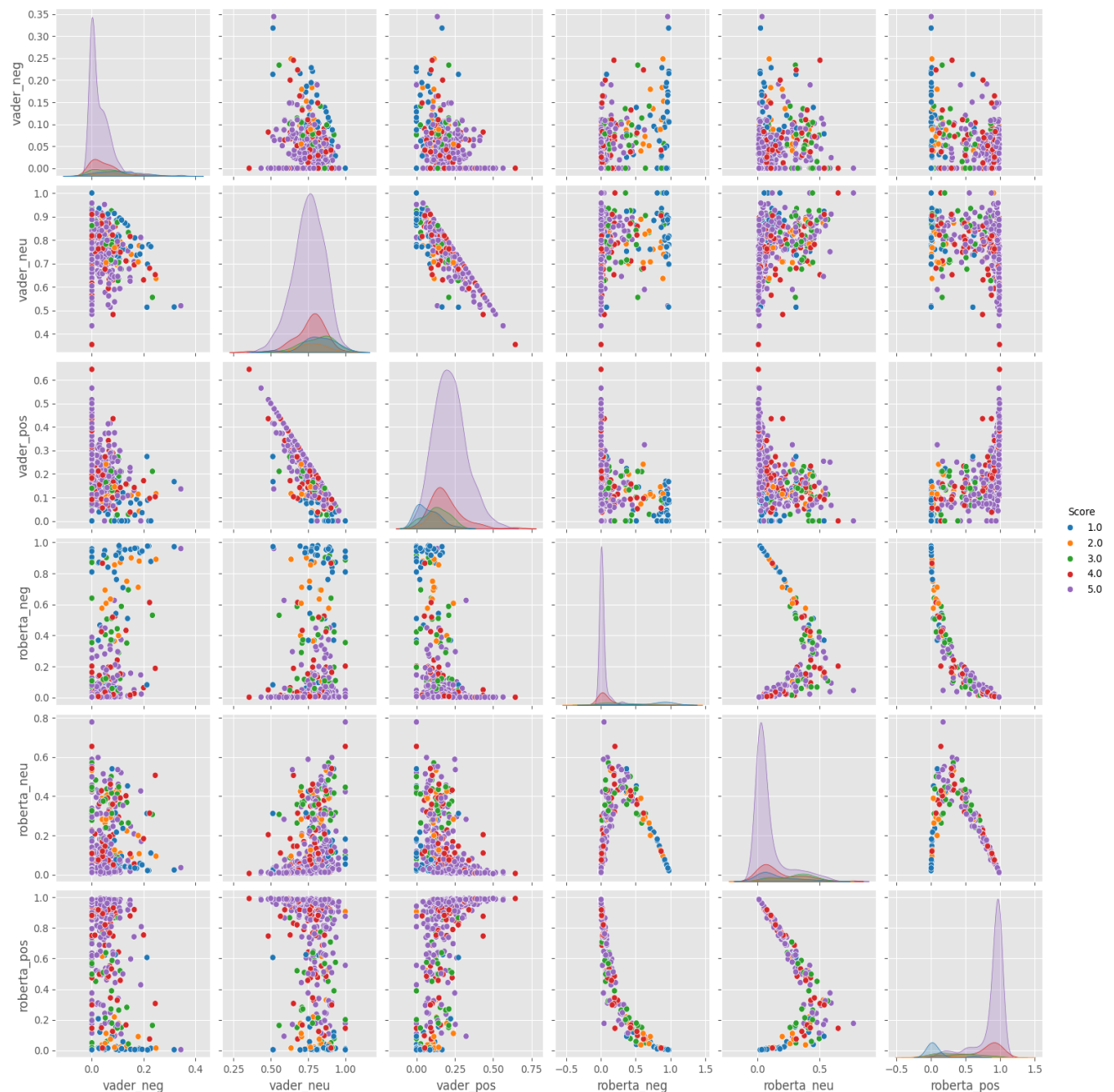
100%  500/500 [04:12<00:00, 1.83it/s]

Broke for id 83
Broke for id 187

```
results_df = pd.DataFrame(res).T
results_df = results_df.reset_index().rename(columns={'index': 'Id'})
results_df = results_df.merge(df, how='left')
```

Compare Results:

Let's see comparison between each observation and what each feature looks like



we can see that there are some correlations between the roberta model and the vader model, it's a little hard to see exactly if there are correlations but one thing that becomes

very clear is that the vader model is a little bit less confident in all of its predictions compared to the roberta model which really separates the positive, neutral and negative scores for each of these predicted values but if we look here this positive and neutral the roberta model has very high scores for the five stars and most of these one stars are very low in positive sentiment scoring.

Review Examples:

Positive 1 star and Negative 5 star reviews

```
results_df.query('Score == 1') \
    .sort_values('roberta_pos', ascending=False)['Text'].values[0]
```

'I felt energized within five minutes, but it lasted for about 45 minutes. I paid \$3.99 for this drink. I could have just drunk a cup of coffee and saved my money.'

```
results_df.query('Score == 1') \
    .sort_values('vader_pos', ascending=False)['Text'].values[0]
```

'So we cancelled the order. It was cancelled without any problem. That is a positive note...'

```
# negative sentiment 5-Star view
results_df.query('Score == 5') \
    .sort_values('roberta_neg', ascending=False)['Text'].values[0]
```

'this was soooooo delicious but too bad i ate em too fast and gained 2 pds! my fault'

```
results_df.query('Score == 5') \
    .sort_values('vader_neg', ascending=False)['Text'].values[0]
```

'this was soooooo delicious but too bad i ate em too fast and gained 2 pds! my fault'

Roberta thinks (for Positive sentiment)

It says I felt energized within five minutes but it lasts about 45 minutes. I paid 3.99 for this drink. I could have just drunk a cup of coffee and save my money. So this is very nuanced sentence and you can see that it starts off being sort of positive I felt energized and it lasted 45 minutes, the model is getting confused and thinking this is more of a positive statement but then we can tell that this is saying negative by the end of the statement.

Vader thinks (for Positive sentiment)

It says so we cancelled the order, it was cancelled without any problem that is a positive note, so they actually used the word positive and without any problem seems positive but it is a negative review and it's being a little sarcastic but the model does not pick on up on that especially the Vader type of model which is only looking at a bag of words.

Roberta thinks (for Negative sentiment)

It says this was so delicious but too bad I ate them too fast and gained two pounds my fault okay so it is sort of a negative sentiment but a positive review.

Vader think (for Negative sentiment)

It happens to be the exact same with Vader also.

So, both models got we could say confused but maybe this actually is a negative sentiment for a positive review so maybe it did a better job than what we would expect to see.

Using the hugging face transformers pipelines:

This just makes everything really simple.

```
from transformers import pipeline
```

```
sent_pipeline = pipeline("sentiment-analysis")
```

No model was supplied, defaulted to distilbert/distilbert-base-uncased-finetuned-sst-2-english and revision af0f99b (<https://huggingface.co/distilbert/distilbert-base-uncased-finetuned-sst-2-english>)
Using a pipeline without specifying a model name and revision in production is not recommended.

```
config.json: 100% ██████████ 629/629 [00:00<00:00, 12.4kB/s]
```

```
model.safetensors: 100% ██████████ 268M/268M [00:01<00:00, 193MB/s]
```

```
tokenizer_config.json: 100% ██████████ 48.0/48.0 [00:00<00:00, 1.38kB/s]
```

```
vocab.txt: 100% ██████████ 232k/232k [00:00<00:00, 4.13MB/s]
```

```
sent_pipeline('I love sentiment analysis!')
```

```
[{'label': 'POSITIVE', 'score': 0.9997853636741638}]
```

```
sent_pipeline('Make sure to like and subscribe!')
```

```
[{'label': 'POSITIVE', 'score': 0.9991742968559265}]
```

```
sent_pipeline('booo')
```

```
[{'label': 'NEGATIVE', 'score': 0.9936267137527466}]
```

Basically, we import from transformers library a pipeline, it will automatically be set up tasks for whichever we want and we want to do sentiment analysis and this will automatically download their default model and embeddings for this pipeline and with this pipeline you can just run sentiment analysis with two lines of code super quick.

Report submitted by

Name: T VamsiKrishna Yadav

Dept: EECS

Roll No: 20287